Name :- Jai Pratap Rajput

CU ID : CU24250071

BTech CSE 3rd Year SECTION - "C"

Roll No. :- 27

==========

Lab Sheet No. 6

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

print("All libraries imported successfully!")
```

```
All libraries imported successfully!
```

```
from google.colab import files

uploaded = files.upload()

file_name = list(uploaded.keys())[0]

df = pd.read_csv(file_name)

print("Dataset loaded successfully!")
print("File name:", file_name)
```

```
Choose Files California_..._Dataset.csv
California_Housing_Dataset.csv(text/csv) - 203374 bytes, last modified: 9/11/2026 - 100% done
Saving California_Housing_Dataset.csv to California_Housing_Dataset.csv
Dataset loaded successfully!
File name: California_Housing_Dataset.csv
```

```
print("First 5 rows of the dataset:")

display(df.head())
```

First 5 rows of the dataset:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVal
0	14.4861	42.76	5.2717	1.2163	1277.51	1.9841	41.2043	-115.6204	500000.00
1	10.2160	51.16	3.7031	0.7862	2257.35	3.9569	34.0654	-119.0866	495653.06
2	15.0000	14.24	7.1359	1.5692	1102.55	2.5094	35.4069	-121.9213	500000.00
3	15.0000	50.46	5.1356	1.0960	4523.25	2.7289	40.6373	-122.3795	500000.00
4	9.6912	22.99	4.9133	1.2624	640.16	4.1603	34.4163	-114.6183	452263.50

```
print("Dataset Shape:")
print(df.shape)

print("\nColumn Names:")
print(df.columns.tolist())

print("\nData Types:")
print(df.dtypes)
```

```
Dataset Shape:
(3000, 9)

Column Names:
['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup', 'Latitude', 'Longitude', 'MedHouseVal']

Data Types:
MedInc        float64
HouseAge      float64
AveRooms      float64
AveBedrms     float64
Population    float64
AveOccup      float64
Latitude      float64
Longitude     float64
MedHouseVal   float64
dtype: object
```

```
print("Missing Values:")
print(df.isnull().sum())

print("\nNumber of Duplicate Rows:")
print(df.duplicated().sum())
```

```
Missing Values:
MedInc        0
HouseAge      0
AveRooms      0
AveBedrms     0
Population    0
AveOccup      0
Latitude      0
Longitude     0
MedHouseVal   0
dtype: int64

Number of Duplicate Rows:
0
```

```
# Remove duplicate rows
df = df.drop_duplicates()

# Fill missing numerical values using median
numeric_columns = df.select_dtypes(include=np.number).columns

df[numeric_columns] = df[numeric_columns].fillna(
    df[numeric_columns].median()
)

print("Data preprocessing completed!")

print("\nMissing values after preprocessing:")
print(df.isnull().sum())
```

```
Data preprocessing completed!

Missing values after preprocessing:
MedInc        0
HouseAge      0
AveRooms      0
AveBedrms     0
Population    0
AveOccup      0
Latitude      0
Longitude     0
MedHouseVal   0
dtype: int64
```

```
print("Statistical Summary of Dataset:")

display(df.describe())
```

Statistical Summary of Dataset:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVal
count	3000.000000	3000.000000	3000.000000	3000.000000	3000.000000	3000.000000	3000.000000	3000.000000	3000.000000
mean	10.901426	26.234883	5.479871	1.095927	1862.964600	3.021457	37.318164	-119.296753	427404.185627
std	3.702997	14.692120	1.244128	0.310180	1378.753659	0.796160	2.701454	2.913618	104347.424234
min	1.853900	1.000000	2.000000	0.500000	170.930000	1.000000	32.554800	-124.348900	25000.000000
25%	7.805725	13.587500	4.635925	0.876650	969.517500	2.474675	34.973050	-121.786800	368524.185000
50%	11.171900	25.890000	5.460850	1.073800	1498.590000	3.021350	37.327300	-119.273750	500000.000000
75%	15.000000	38.862500	6.338625	1.289875	2324.225000	3.565700	39.626625	-116.843075	500000.000000
max	15.000000	51.970000	9.734900	2.324300	13749.970000	6.583300	41.944800	-114.153100	500000.000000

```
correlation = df.corr(numeric_only=True)

print("Correlation Matrix:")

display(correlation)
```

Correlation Matrix:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVal
MedInc	1.000000	0.012248	0.004154	-0.003563	-0.019127	-0.023399	-0.011700	0.011650	0.864742
HouseAge	0.012248	1.000000	-0.010989	-0.017002	0.021150	0.002802	0.003029	-0.004803	0.006213
AveRooms	0.004154	-0.010989	1.000000	0.792917	0.029065	-0.004320	0.012132	-0.003618	-0.004171
AveBedrms	-0.003563	-0.017002	0.792917	1.000000	0.016545	-0.016952	0.009740	0.009576	-0.007480
Population	-0.019127	0.021150	0.029065	0.016545	1.000000	0.000441	-0.026138	0.012833	-0.160284
AveOccup	-0.023399	0.002802	-0.004320	-0.016952	0.000441	1.000000	-0.008936	-0.012685	-0.040464
Latitude	-0.011700	0.003029	0.012132	0.009740	-0.026138	-0.008936	1.000000	-0.003129	-0.026284
Longitude	0.011650	-0.004803	-0.003618	0.009576	0.012833	-0.012685	-0.003129	1.000000	-0.001926
MedHouseVal	0.864742	0.006213	-0.004171	-0.007480	-0.160284	-0.040464	-0.026284	-0.001926	1.000000

Next steps: Generate code with `correlation` New interactive sheet

```
plt.figure(figsize=(10, 7))

plt.imshow(
    correlation,
    cmap='coolwarm',
    interpolation='nearest'
)

plt.colorbar()

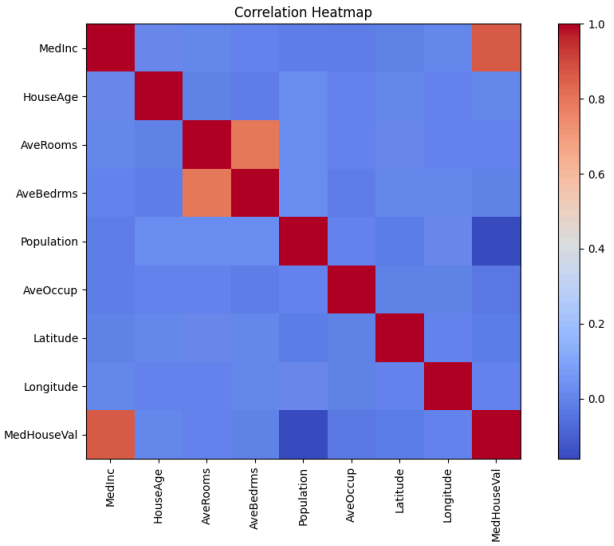
plt.xticks(
    range(len(correlation.columns)),
    correlation.columns,
    rotation=90
)

plt.yticks(
    range(len(correlation.columns)),
    correlation.columns
)

plt.title("Correlation Heatmap")

plt.tight_layout()

plt.show()
```




```
X = df.drop('MedHouseVal', axis=1)

y = df['MedHouseVal']

print("Independent Variables:")
print(X.columns.tolist())

print("\nDependent Variable:")
print("MedHouseVal")
```

```
Independent Variables:
['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup', 'Latitude', 'Longitude']

Dependent Variable:
MedHouseVal
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("Training Data Shape:", X_train.shape)
print("Testing Data Shape :", X_test.shape)
```

```
Training Data Shape: (2400, 8)
Testing Data Shape : (600, 8)
```

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()

print("Linear Regression model created successfully!")
```

```
Linear Regression model created successfully!
```

```
model.fit(X_train, y_train)

print("Linear Regression model trained successfully!")
```

```
Linear Regression model trained successfully!
```

```
y_pred = model.predict(X_test)

print("Predictions generated successfully!")

print("\nFirst 10 Actual vs Predicted Values:")

comparison = pd.DataFrame({
    'Actual Value': y_test.values[:10],
    'Predicted Value': y_pred[:10]
})

display(comparison)
```

```
Predictions generated successfully!

First 10 Actual vs Predicted Values:
```

	Actual Value	Predicted Value
0	500000.00	438426.018869
1	500000.00	455595.321518
2	299817.64	351767.520166
3	500000.00	530162.095271
4	405462.33	329519.959720
5	489454.87	429016.339359
6	500000.00	526016.495866
7	325040.79	328521.360994
8	500000.00	448142.549021
9	500000.00	540539.341711

Next steps: Generate code with `comparison` New interactive sheet

```
mae = mean_absolute_error(y_test, y_pred)

print(f"Mean Absolute Error (MAE): {mae:.2f}")
```

```
Mean Absolute Error (MAE): 39814.55
```

```
mse = mean_squared_error(y_test, y_pred)

print(f"Mean Squared Error (MSE): {mse:.2f}")
```

```
Mean Squared Error (MSE): 2446536198.33
```

```
rmse = np.sqrt(mse)

print(f"Root Mean Squared Error (RMSE): {rmse:.2f}")
```

```
Root Mean Squared Error (RMSE): 49462.47
```

```
r2 = r2_score(y_test, y_pred)

print(f"R² Score: {r2:.4f}")
```

```
R² Score: 0.7605
```

```
metrics = pd.DataFrame({
    'Metric': [
        'MAE',
        'MSE',
        'RMSE',
        'R² Score'
    ],
    'Value': [
        mae,
        mse,
        rmse,
        r2
    ]
})

display(metrics)
```

	Metric	Value
0	MAE	3.981455e+04
1	MSE	2.446536e+09
2	RMSE	4.946247e+04
3	R² Score	7.605400e-01

Next steps: Generate code with `metrics` New interactive sheet

```
plt.figure(figsize=(10, 6))

plt.scatter(
    y_test,
    y_pred,
    alpha=0.6
)

plt.xlabel("Actual House Value")
plt.ylabel("Predicted House Value")

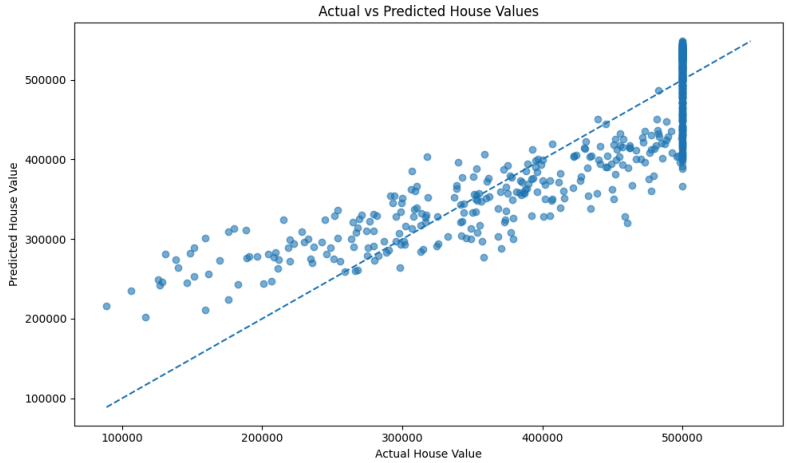
plt.title("Actual vs Predicted House Values")

# Perfect prediction reference line
minimum = min(y_test.min(), y_pred.min())
maximum = max(y_test.max(), y_pred.max())

plt.plot(
    [minimum, maximum],
    [minimum, maximum],
    linestyle='--'
)

plt.tight_layout()

plt.show()
```




```
plt.figure(figsize=(10, 6))

plt.scatter(
    df['MedInc'],
    df['MedHouseVal'],
    alpha=0.5
)

# Fit a simple regression line for visualization
x = df[['MedInc']]
y_value = df['MedHouseVal']

simple_model = LinearRegression()
simple_model.fit(x, y_value)

x_line = np.linspace(
    df['MedInc'].min(),
    df['MedInc'].max(),
    100
).reshape(-1, 1)

y_line = simple_model.predict(x_line)

plt.plot(
    x_line,
    y_line,
    linestyle='--',
    linewidth=2
)

plt.xlabel("Median Income")
plt.ylabel("Median House Value")

plt.title(
    "Regression Line: Median Income vs Median House Value"
)

plt.tight_layout()

plt.show()
```

```
/usr/local/lib/python3.13/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but LinearRegression was fitted with feature names
  warnings.warn(
```




```
coefficients = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': model.coef_
})

coefficients = coefficients.sort_values(
    'Coefficient',
    ascending=False
)

display(coefficients)
```

	Feature	Coefficient
0	MedInc	24409.948368
2	AveRooms	26.368772
4	Population	-11.159458
1	HouseAge	-59.600605
7	Longitude	-355.287616
3	AveBedrms	-635.536929
6	Latitude	-757.252318
5	AveOccup	-2892.847529

Next steps: Generate code with `coefficients` New interactive sheet

```
plt.figure(figsize=(10, 6))

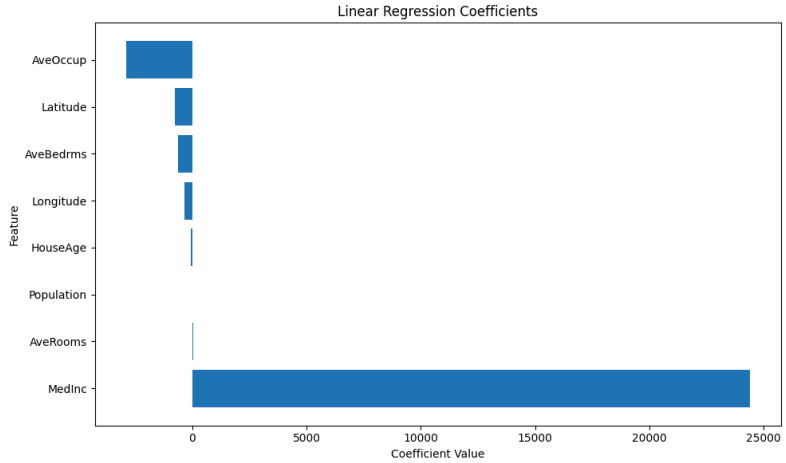
plt.barh(
    coefficients['Feature'],
    coefficients['Coefficient']
)

plt.xlabel("Coefficient Value")
plt.ylabel("Feature")

plt.title("Linear Regression Coefficients")

plt.tight_layout()

plt.show()
```




```
positive_feature = coefficients.iloc[0]
negative_feature = coefficients.iloc[-1]

print("Feature with the strongest positive coefficient:")
print(
    positive_feature['Feature'],
    "->",
    round(positive_feature['Coefficient'], 2)
)

print("\nFeature with the strongest negative coefficient:")
print(
    negative_feature['Feature'],
    "->",
    round(negative_feature['Coefficient'], 2)
)
```

```
Feature with the strongest positive coefficient:
MedInc -> 24409.95

Feature with the strongest negative coefficient:
AveOccup -> -2892.85
```

```
print("Model Intercept:")
print(model.intercept_)
```

```
Model Intercept:
178762.52629780234
```

```
results = pd.DataFrame({
    'Actual': y_test,
    'Predicted': y_pred
})

results = results.reset_index(drop=True)

display(results.head(20))
```

	Actual	Predicted
0	500000.00	438426.018869
1	500000.00	455595.321518
2	299817.64	351767.520166
3	500000.00	530162.095271
4	405462.33	329519.959720
5	489454.87	429016.339359
6	500000.00	526016.495866
7	325040.79	328521.360994
8	500000.00	448142.549021
9	500000.00	540539.341711
10	179714.20	313163.580789
11	362154.48	353279.008783
12	358423.53	406214.263228
13	148390.98	282027.362475
14	422194.55	403951.005474
15	500000.00	507075.319949
16	300882.11	297711.074190
17	275153.82	278985.195621
18	500000.00	522306.339373
19	345356.31	300809.817358

```
results['Error'] = (
    results['Actual'] -
    results['Predicted']
)

results['Absolute Error'] = (
    results['Error'].abs()
)

display(results.head(20))
```

	Actual	Predicted	Error	Absolute Error
0	500000.00	438426.018869	61573.981131	61573.981131
1	500000.00	455595.321518	44404.678482	44404.678482
2	299817.64	351767.520166	-51949.880166	51949.880166
3	500000.00	530162.095271	-30162.095271	30162.095271
4	405462.33	329519.959720	75942.370280	75942.370280
5	489454.87	429016.339359	60438.530641	60438.530641
6	500000.00	526016.495866	-26016.495866	26016.495866
7	325040.79	328521.360994	-3480.570994	3480.570994
8	500000.00	448142.549021	51857.450979	51857.450979
9	500000.00	540539.341711	-40539.341711	40539.341711
10	179714.20	313163.580789	-133449.380789	133449.380789
11	362154.48	353279.008783	8875.471217	8875.471217
12	358423.53	406214.263228	-47790.733228	47790.733228
13	148390.98	282027.362475	-133636.382475	133636.382475
14	422194.55	403951.005474	18243.544526	18243.544526
15	500000.00	507075.319949	-7075.319949	7075.319949
16	300882.11	297711.074190	3171.035810	3171.035810
17	275153.82	278985.195621	-3831.375621	3831.375621
18	500000.00	522306.339373	-22306.339373	22306.339373
19	345356.31	300809.817358	44546.492642	44546.492642

```
plt.figure(figsize=(10, 6))

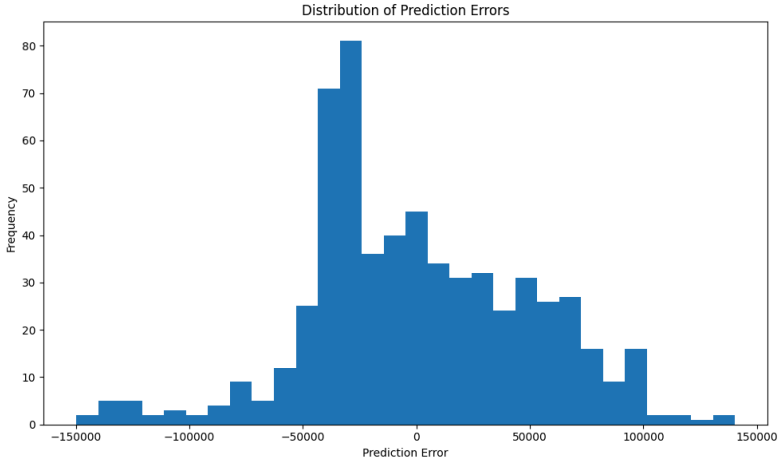
plt.hist(
    results['Error'],
    bins=30
)

plt.xlabel("Prediction Error")
plt.ylabel("Frequency")

plt.title("Distribution of Prediction Errors")

plt.tight_layout()

plt.show()
```




```
print("================================================")
print("       LINEAR REGRESSION PERFORMANCE")
print("================================================")

print(f"Mean Absolute Error (MAE)  : {mae:,.2f}")
print(f"Mean Squared Error (MSE)   : {mse:,.2f}")
print(f"Root Mean Squared Error    : {rmse:,.2f}")
print(f"R² Score                   : {r2:.4f}")

print("================================================")
```

```
================================================
       LINEAR REGRESSION PERFORMANCE
================================================
Mean Absolute Error (MAE)  : 39,814.55
Mean Squared Error (MSE)   : 2,446,536,198.33
Root Mean Squared Error    : 49,462.47
R² Score                   : 0.7605
================================================
```

```
print("================================================")
print("            MODEL INTERPRETATION")
print("================================================")

print(f"""
1. The model uses multiple independent variables
   to predict median house value.

2. The R² Score is {r2:.4f}, which indicates how much
   variation in house values is explained by the model.

3. The average absolute prediction error is approximately
   ${mae:,.2f}.

4. The RMSE is approximately ${rmse:,.2f}.

5. The feature with the strongest positive coefficient is
   {positive_feature['Feature']}.

6. The feature with the strongest negative coefficient is
   {negative_feature['Feature']}.

7. Linear Regression is suitable when the relationship
   between predictors and the target is approximately linear.
""")
```

```
================================================
            MODEL INTERPRETATION
================================================

1. The model uses multiple independent variables
   to predict median house value.
```

```
2. The R² Score is 0.7605, which indicates how much
   variation in house values is explained by the model.

3. The average absolute prediction error is approximately
   $39,814.55.

4. The RMSE is approximately $49,462.47.

5. The feature with the strongest positive coefficient is
   MedInc.

6. The feature with the strongest negative coefficient is
   AveOccup.

7. Linear Regression is suitable when the relationship
   between predictors and the target is approximately linear.
```

```
print("================================================")
print("                  CONCLUSION")
print("================================================")

print("""
The Linear Regression model was successfully developed
to predict median house values.

The dataset was preprocessed, divided into training and
testing sets, and used to train the regression model.

The model generated predictions for the test dataset and
was evaluated using MAE, MSE, RMSE, and R² Score.

Regression coefficients were also analyzed to understand
the influence of different independent variables.

Overall, the experiment demonstrates how Linear Regression
can be used for predictive analytics and continuous-value
prediction.
""")
```

```
================================================
                  CONCLUSION
================================================

The Linear Regression model was successfully developed
to predict median house values.

The dataset was preprocessed, divided into training and
testing sets, and used to train the regression model.

The model generated predictions for the test dataset and
was evaluated using MAE, MSE, RMSE, and R² Score.

Regression coefficients were also analyzed to understand
the influence of different independent variables.

Overall, the experiment demonstrates how Linear Regression
can be used for predictive analytics and continuous-value
prediction.
```

Q1. What is Predictive Analytics, and how is it used in real-world applications?

Ans - Predictive Analytics is the process of using historical data, statistical techniques, and machine learning algorithms to predict future outcomes. It helps organizations identify patterns and trends in existing data and use them to make predictions. In real-world applications, predictive analytics is used for house price prediction, sales forecasting, customer behavior prediction, risk assessment, demand forecasting, and financial analysis.

Q2. Explain the working principle of the Linear Regression algorithm.

Ans - Linear Regression is a supervised machine learning algorithm used to predict a continuous numerical value. It establishes a relationship between one or more independent variables and a dependent variable by fitting a linear equation to the data. The model finds suitable coefficients that represent the relationship between the input features and the target value and uses this equation to predict values for new data.

Q3. Differentiate between dependent and independent variables with suitable examples.

Ans - An independent variable is an input feature used to make a prediction, whereas a dependent variable is the output or target that the model tries to predict. For example, in house price prediction, median income, house age, number of rooms, and population are independent variables, while median house value is the dependent variable. Thus, independent variables are used as inputs and the dependent variable is the predicted outcome.

Q4. Why is it necessary to split the dataset into training and testing sets?

Ans - Splitting the dataset into training and testing sets is necessary to evaluate how well a machine learning model performs on unseen data. The training dataset is used to train the model and learn relationships between the input features and target variable, while the testing dataset is used to evaluate the trained model. A common approach is to use 80% of the data for training and 20% for testing.

Q5. What is the significance of the R^2 Score in regression analysis?

Ans - R^2 Score, also called the coefficient of determination, measures how well a regression model explains the variation in the dependent variable. A higher R^2 value generally indicates that the model explains a greater proportion of the variation in the target variable. It is commonly used to measure the goodness of fit of a regression model.

Q6. Differentiate between MAE, MSE, and RMSE. Which metric is more sensitive to large prediction errors?

Ans - MAE, MSE, and RMSE are metrics used to measure the performance of regression models. MAE or Mean Absolute Error calculates the average absolute difference between actual and predicted values. MSE or Mean Squared Error calculates the average of the squared errors, so large errors have a greater effect. RMSE or Root Mean Squared Error is the square root of MSE and represents the error in the same units as the target variable. MSE and RMSE are more sensitive to large prediction errors because they involve squared errors.

Q7. What assumptions should be satisfied before applying Linear Regression?

Ans - Before applying Linear Regression, important assumptions include a linear relationship between the independent and dependent variables, independence of observations, constant variance of errors, approximately normally distributed residuals, and low multicollinearity among independent variables. Meeting these assumptions helps produce more reliable and meaningful regression results.

Q8. How can overfitting and underfitting affect the performance of a regression model?

Ans - Overfitting occurs when a model learns the training data too closely, including noise and unnecessary patterns. As a result, it performs well on training data but poorly on unseen data. Underfitting occurs when the model is too simple to capture important relationships in the data, resulting in poor performance on both training and testing data. A good model should maintain a proper balance between overfitting and underfitting.

Q9. Mention any three real-world applications of Linear Regression in business or industry.

Ans - Three real-world applications of Linear Regression are house price prediction, sales forecasting, and demand forecasting. In house price prediction, property-related features can be used to estimate prices. In sales forecasting, historical sales data can be used to predict future sales. In demand forecasting, factors such as previous demand, price, and season can be used to estimate future product demand.

Q10. How can feature selection improve the accuracy and interpretability of a predictive model?

Ans - Feature selection is the process of choosing the most relevant input variables for a predictive model. It can improve accuracy by removing irrelevant or redundant features that may introduce noise. It can also reduce multicollinearity and make the model easier to understand. Using fewer and more relevant features helps make the effect of each variable on the target clearer and improves the interpretability of the model.